

■ 복합 연산자

- 복합 연산자: 자료형에 적용하는 기본 연산자와 = 연산자를 함께 사용해 구성하는 연산자
- $a += 10$ 이라고 사용하면 $a = a + 10$ 이라고 하는 것과 같은 결과를 냄
- 숫자에 적용할 수 있는 다른 연산자들도 같은 방법으로 사용

표 2-19 숫자에 적용하는 복합 대입 연산자

연산자	설명
$+=$	숫자 덧셈 후 대입 연산자
$-=$	숫자 뺄셈 후 대입 연산자
$*=$	숫자 곱셈 후 대입 연산자
$/=$	숫자 나눗셈 후 대입 연산자
$\%=$	숫자 나머지 연산 후 대입 연산자

■ 복합 연산자

- 문자열도 마찬가지로 +(문자열 연결 연산자)와 = 연산자를 합쳐 [표 2-20]의 복합 대입 연산자를 사용할 수 있음

표 2-20 문자열에 적용하는 복합 대입 연산자

연산자	설명
<code>+=</code>	문자열 연결 후 대입 연산자

■ 복합 연산자

- **기본예제 2-17** 숫자와 관련된 복합 대입 연산자/2장/AssignmentOperator
 - 정수 자료형의 변수 output을 만들고 복합 대입 연산자를 적용

코드 2-37 숫자와 관련된 복합 대입 연산자

```
01 static void Main(string[] args)
02 {
03     int output = 0;
04     output += 52;
05     output += 273;
06     output += 103;
07
08     Console.WriteLine(output);
09 }
```

실행 결과

428

■ 복합 연산자

- **기본예제 2-17** 숫자와 관련된 복합 대입 연산자/2장/AssignmentOperator

코드 2-38

숫자와 관련된 복합 대입 연산자의 다른 방식

```
01 static void Main(string[] args)
02 {
03     int output = 0;
04     output = output + 52;
05     output = output + 273;
06     output = output + 103;
07
08     Console.WriteLine(output);
09 }
```



Microsoft Visual Studio 디버그 ×

05 복합 대입 연산자

■ 복합 연산자

- **기본예제 2-18** 문자열과 관련된 복합 대입 연산자 [/2장/StringAssignmentOperator](#)
 - 문자열 자료형의 변수 output을 만들고 복합 대입 연산자를 사용

코드 2-39 문자열과 관련된 복합 대입 연산자

```
01 static void Main(string[] args)
02 {
03     string output = "hello ";
04     output += "world ";
05     output += "!";
06
07     Console.WriteLine(output);
08 }
```

실행 결과

hello world !

05 복합 대입 연산자

■ 복합 연산자

- **기본예제 2-18** 문자열과 관련된 복합 대입 연산자 [/2장/StringAssignmentOperator](#)

코드 2-40

문자열과 관련된 복합 대입 연산자 예제 풀어쓰기

```
01 static void Main(string[] args)
02 {
03     string output = "hello ";
04     output = output + "world ";
05     output = output + "!";
06
07     Console.WriteLine(output);
08 }
```



Microsoft Visual Studio 디버그



hello world !

■ 증감 연산자

- 증감 연산자는 단항 연산자로 변수앞과 뒤에 ++ 기호와 -- 기호를 붙여 만듦

표 2-21 증감 연산자

연산자	설명
[변수]++	기존 변수의 값에 1을 더합니다(후위).
++[변수]	기존 변수의 값에 1을 더합니다(전위).
[변수]--	기존 변수의 값에서 1을 뺍니다(후위).
--[변수]	기존 변수의 값에서 1을 뺍니다(전위).

■ 증감 연산자

■ 기본예제 2-19 증감 연산자

/2장/IncrementOperator

- 변수 number를 초기화하고 ++ 연산자와 -- 연산자를 사용

코드 2-41 증감 연산자

```
01 static void Main(string[] args)
02 {
03     int number = 10;
04     number++;
05     Console.WriteLine(number);
06     number--;
07     Console.WriteLine(number);
08 }
```

실행 결과

```
11
10
```

- 한 줄에 독립적으로 증감 연산자를 사용할 때는 전위와 후위의 차이를 알 수 없음

06 증감 연산자

■ 증감 연산자

- **기본예제 2-20** 증감 연산자의 전위와 후위 [/2장/IncrementOperatorPosition](#)
 - 증감 연산자의 전위 형태와 후위 형태를 사용

코드 2-42 증감 연산자의 후위 형태

```
01 static void Main(string[] args)
02 {
03     int number = 10;
04     Console.WriteLine(number);
05     Console.WriteLine(number++);
06     Console.WriteLine(number--);
07     Console.WriteLine(number);
08 }
```

실행 결과

```
10
10
11
10
```

06 증감 연산자

■ 증감 연산자

- **기본예제 2-20** 증감 연산자의 전위와 후위 [/2장/IncrementOperatorPosition](#)

코드 2-43 증감 연산자의 전위 형태

```
01 static void Main(string[] args)
02 {
03     int number = 10;
04     Console.WriteLine(number);

05     Console.WriteLine(++number);
06     Console.WriteLine(--number);
07     Console.WriteLine(number);
08 }
```

실행 결과

```
10
11
10
10
```

■ 증감 연산자

- **기본예제 2-20** 증감 연산자의 전위와 후위 [/2장/IncrementOperatorPosition](#)
 - 후위는 문장을 실행한 이후에 값을 변경하라는 의미
 - `Console.WriteLine(number++)` 코드는 `Console.WriteLine(number)`를 실행하고 변수 `number`에 1을 더함

코드 2-44

후위 증감 연산자

[/2장/Operators](#)

```
01 static void Main(string[] args)
02 {
03     int number = 10;
04     Console.WriteLine(number);
05     Console.WriteLine(number); number += 1; —— Console.WriteLine(number++)
06     Console.WriteLine(number); number -= 1; —— Console.WriteLine(number--)
07     Console.WriteLine(number);
08 }
```

■ 증감 연산자

■ 기본예제 2-20 증감 연산자의 전위와 후위 [/2장/IncrementOperatorPosition](#)

- 해당 문장을 실행하기 전에 값을 더하는 것이 전위

코드 2-45 증감 연산자 이해도 확인

[/2장/Operators](#)

```
01 static void Main(string[] args)
02 {
03     int number = 10;
04     Console.WriteLine(number++);
05     Console.WriteLine(++number);
06     Console.WriteLine(number--);
07     Console.WriteLine(--number);
08 }
```

- 코드를 실행하면 차례대로 10, 12, 12, 10을 출력

■ 증감 연산자

■ 기본예제 2-20 증감 연산자의 전위와 후위 [/2장/IncrementOperatorPosition](#)

- 특별한 이유가 아니라면 한 줄에 하나의 증감 연산자만 독립적으로 사용하기 바람

코드 2-46

여러 줄로 나누어 적은 증감 연산자

[/2장/Operators](#)

```
01 static void Main(string[] args)
02 {
03     int number = 10;
04     Console.WriteLine(number);
05     number++;
06     number++;
07     Console.WriteLine(number);
08     Console.WriteLine(number);
09     number--;
10     number--;
11     Console.WriteLine(number);
12 }
```

Microsoft Visual Studio 디버그

10
12
12
10

■ 자료형 출력

- 자료형을 확인하는 가장 쉬운 방법은 해당 변수에 마우스 커서를 가져다 대는 것

```
int number = 10;  
Console.WriteLine(number);
```

[?] (지역 변수) int number

그림 2-15 마우스를 사용한 자료형 확인

- 프로그램 내부에서 자료형을 출력하고 싶은 경우에는 [표 2-22]의 GetType() 메서드를 사용

표 2-22 자료형 확인 메서드

메서드	설명
GetType()	해당 변수의 자료형을 추출합니다.

■ 자료형 출력

■ 기본예제 2-21 GetType() 메서드 활용

/2장/TypeCheck

- GetType() 메서드를 활용해서 자료형을 출력

코드 2-47 GetType() 메서드 활용

```
01 static void Main(string[] args)
02 {
03     // 변수를 선언합니다.
04     int _int = 273;
05     long _long = 522731033265;
06     float _float = 52.273F;
07     double _double = 52.273;
08
09     char _char = '글';
10     string _string = "문자열";
11
12     // 출력합니다.
13     Console.WriteLine(_int.GetType());
14     Console.WriteLine(_long.GetType());
15     Console.WriteLine(_float.GetType());
16     Console.WriteLine(_double.GetType());
17     Console.WriteLine(_char.GetType());
18     Console.WriteLine(_string.GetType());
19 }
```

실행 결과

```
System.Int32
System.Int64
System.Single
System.Double
System.Char
System.String
```

07 자료형 검사

■ 자료형 출력

- **기본예제 2-22** 직접적인 GetType() 메서드 활용 [/2장/DirectTypeCheck](#)
 - 숫자 또는 문자열에 직접 적용할 수도 있음

코드 2-48 직접적인 GetType() 메서드 활용

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine((273).GetType());
04     Console.WriteLine((522731033265L).GetType());
05     Console.WriteLine((52.273F).GetType());
06     Console.WriteLine((52.273).GetType());
07     Console.WriteLine(('자').GetType());
08     Console.WriteLine(("문자열").GetType());
09 }
```

실행 결과

```
System.Int32
System.Int64
System.Single
System.Double
System.Char
System.String
```


■ 자료형 지정

- C#에서는 [표 2-23]의 var 키워드로 변수의 자료형을 자동으로 지정

표 2-23 var 키워드

키워드	설명
var	자료형을 자동으로 지정합니다.

- 변수를 선언하면 초기화할 때에 자료형이 자동 지정

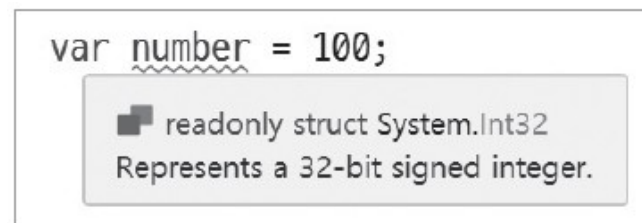


그림 2-16 var 키워드로 자료형 자동 결정

■ 자료형 지정

- int 자료형으로 선언된 변수를 string 자료형으로 바꾸거나 하는 것은 불가능

코드 2-50

var 키워드의 제약

/2장/VarKeyword

```
01 var number = 100;  
02 number = "변경";
```

```
var number = 100;  
number = "변경";
```

 class System.String
Represents text as a sequence of UTF-16 code units.

CS0029: 암시적으로 'string' 형식을 'int' 형식으로 변환할 수 없습니다.

그림 2-17 var 키워드 제약

■ 자료형 지정

- var 키워드를 사용하려면 다음과 같은 두 가지 조건을 만족해야 함
 - 지역 변수로 선언하는 경우
 - 변수를 선언과 동시에 초기화하는 경우

■ 자료형 지정

- 지역변수: C#에서는 메서드 내부에 선언한 변수

코드 2-51

지역 변수

/2장/VarKeyword

```
01 class Program
02 {
03     var global = 52;————인스턴스 변수(var 키워드 사용 불가)입니다.
04
05     static void Main(string[] args)
06     {
07         var local = 273;————지역 변수(var 키워드 사용 가능)입니다.
08     }
09 }
```

- 메서드 내부에 선언할 때에만 var 키워드를 적용할 수 있음

■ 자료형 지정

- 메서드 외부에 var 키워드를 사용하면 다음과 같이 오류가 발생

```
class Program
{
    var global = 52;
    static void Main(string[] args)
    {
    }
}
```

참고: CS0825: 상황별 키워드 'var'는 지역 변수 선언이나 스크립트 코드에만 표시할 수 있습니다.

그림 2-18 지역 변수가 아닌 곳에는 var 키워드 사용 불가

■ 자료형 지정

- var 키워드를 다음과 같이 덩그러니 써놓으면 오류가 발생

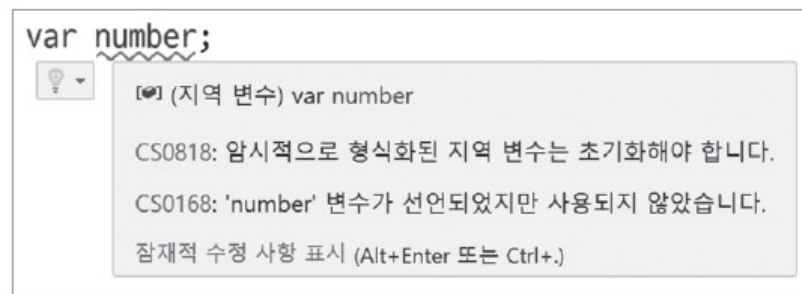


그림 2-19 var 키워드는 선언과 초기화를 함께 작성

- 다음과 같이 선언과 초기화를 함께 해야 함

코드 2-52

var 키워드의 선언과 초기화 동시 수행

/2장/VarKeyword

```
01 static void Main(string[] args)
02 {
03     var number = 20;
04 }
```

■ var 키워드로 long 자료형과 float 자료형인 변수 생성

- long 자료형, float 자료형으로 선언하고 싶다면 숫자 뒤에 L 또는 F 등의 기호를 붙여 명시적으로 해당 숫자가 어떤 자료형인지 알려줘야 함

코드 2-53

var 키워드를 사용한 다양한 자료형 선언

/2장/VarKeyword

```
01 static void Main(string[] args)
02 {
03     var numberA = 100L;        // long 자료형
04     var numberB = 100.0;       // double 자료형
05     var numberC = 100.0F;      // float 자료형
06 }
```

여기서 잠깐!

■ L

코드 2-54

long 자료형을 나타내는 기호: 소문자

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(123456 + 65432l);
04 }
```

실행 결과

188888

- 오른쪽의 숫자가 654321이 아니라 65432L
- 코드를 작성할 때는 다음과 같이 long 자료형임을 나타내는 L은 대문자로 사용하기 바람

여기서 잠깐!

■ L

코드 2-55 long 자료형을 나타내는 기호: 대문자

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(123456 + 65432L);
04 }
```

- 비주얼 스튜디오에서 l을 소문자로 사용하면 다음과 같은 경고도 뜬

65432l;

readonly struct System.Int64
Represents a 64-bit signed integer.

CS0078: 접미사 'l'은 숫자 '1'과 쉽게 혼동됩니다. 쉽게 구별할 수 있도록 'L'을 사용하세요.

그림 2-20 소문자 경고

■ 입력

- C#에서 사용자의 입력을 받을 때는 [표 2-24]의 메서드를 사용

표 2-24 입력 메서드

메서드	설명
Console.ReadLine()	사용자로부터 한 줄의 문자열을 입력 받습니다.

- ReadLine() 메서드를 살펴보면 [그림 2-21]
- ReadLine() 메서드의 결과가 string? 자료형으로 나온다는 이야기

```
Console.ReadLine()  
string? Console.ReadLine()  
표준 입력 스트림에서 다음 줄의 문자를 읽습니다.
```

그림 2-21 입력 메서드의 반환형

■ 입력

■ 기본예제 2-23 문자열 입력과 출력

/2장/Input

- 결과를 string 자료형의 변수 input에 넣어주고 입력 받은 문자열을 출력
- 코드를 실행한 다음 아무것이나 입력하고 엔터를 입력

코드 2-56 문자열 입력과 출력

```
01 static void Main(string[] args)
02 {
03     string input = Console.ReadLine();
04     Console.WriteLine("input: " + input);
05 }
```

실행 결과

아무것이나 입력

input: 아무것이나 입력

10 자료형 변환

■ 자료형 변환(Type Casting): 한 자료형을 다른 자료형으로 바꾸는 것

■ 강제 자료형 변환

- 큰 자료형에서 작은 자료형으로 변환될 때는 데이터가 깨질 수 있음

코드 2-57

자료형 변환

/2장/Casts

```
01 static void Main(string[] args)
02 {
03     // long 자료형을 int 자료형으로 변환합니다.
04     long longNumber = 2147483647L + 2147483647L;
05     int intNumber = longNumber;
06     Console.WriteLine(intNumber);
07 }
```

10 자료형 변환

■ 강제 자료형 변환

- long 자료형에 4294967294라는 숫자를 넣고 이를 int 자료형에 넣으려고 함
- int 자료형의 한계 수치는 2147483647이기 때문에 [그림 2-22]와 같은 오류를 냄

```
static void Main(string[] args)
{
    // long 자료형을 int 자료형으로 변환합니다.
    long longNumber = 2147483647L + 2147483647L;
    int intNumber = longNumber;
    Console.WriteLine(
}
```

(지역 변수) long longNumber

CS0266: 암시적으로 'long' 형식을 'int' 형식으로 변환할 수 없습니다. 명시적 변환이 있습니다. 캐스트가 있는지 확인하세요.

잠재적 수정 사항 표시 (Alt+Enter 또는 Ctrl+.)

그림 2-22 자료형 변환 실패

■ 강제 자료형 변환

- 자동으로 변환되지 않는다는 오류가 뜨지만 강제로 이를 변환할 수 있음
- 강제 자료형 변환: 강제로 자료형을 변환하는 것

코드 2-58

강제 자료형 변환

/2장/Casts

```
01 var a = (int)10.0;  
02 var b = (float)10;  
03 var c = (double)10;
```

10 자료형 변환

■ 강제 자료형 변환

■ 기본예제 2-24 강제 자료형 변환

/2장/ExplicitConversion

- long 자료형을 int 자료형으로 변환

코드 2-59 강제 자료형 변환

```
01 static void Main(string[] args)
02 {
03     // long 자료형을 int 자료형으로 변환합니다.
04     long longNumber = 2147483647L + 2147483647L;
05     int intNumber = (int)longNumber;
06     Console.WriteLine(intNumber);
07 }
```

실행 결과

-2

■ 강제 자료형 변환

■ 기본예제 2-24 강제 자료형 변환

/2장/ExplicitConversion

- 데이터 손실: 큰 자료형에서 작은 자료형으로 변환할 때 숫자가 넘쳐 없어질 수 있는 현상
- 다음과 같은 코드는 강제로 자료형을 변환해도 값이 충분히 수용되므로 데이터 손실이 일어나지 않음

코드 2-60

강제 자료형 변환의 데이터 손실 미발생

/2장/Casts

```
01 static void Main(string[] args)
02 {
03     // long 자료형을 int 자료형으로 변환합니다.
04     long longNumber = 52273;
05     int intNumber = (int)longNumber;
06     Console.WriteLine(intNumber);
07 }
```


10 자료형 변환

■ 자동 자료형 변환

- C#은 데이터 손실이 일어나지 않는 범위에 한해서 자동으로 자료형을 변경

■ 기본예제 2-25 숫자 손상

/2장/NumberLost

- 큰 자료

코드 2-61

숫자 손상

```
01 static void Main(string[] args)
02 {
03     // long 자료형을 int 자료형으로 변환합니다.
04     long longNumber = 2147483647L + 2147483647L;
05     int longToInt = (int)longNumber;
06     Console.WriteLine(longToInt);
07
08     // double 자료형을 int 자료형으로 변환합니다.
09     double doubleNumber = 52.27310332;
10     int doubleToInt = (int)doubleNumber;
11     Console.WriteLine(doubleToInt);
12 }
```

실행 결과

-2
52

10 자료형 변환

■ 자동 자료형 변환

- int 자료형의 숫자는 long 자료형에 넣어도 절대 깨지지 않고 자동 자료형 변환이 일어남

표 2-25 자동 자료형 변환

기존 자료형	자동 변환되는 자료형
int	long, float, double
long	float, double
char	int, long, float, double
float	double

10 자료형 변환

■ 자동 자료형 변환

■ 기본예제 2-26 자동 자료형 변환

/2장/ImplicitConversion

- int 자료형을 long과 double 자료형으로 자동 자료형 변환

코드 2-62 자동 자료형 변환

```
01 static void Main(string[] args)
02 {
03     // int 자료형의 숫자를 생성합니다.
04     int intNumber = 2147483647;
05
06     // int 자료형을 long 자료형으로 자동 변환합니다.
07     long intToLong = intNumber;
08     Console.WriteLine(intToLong);
09
10     // int 자료형을 double 자료형으로 자동 변환합니다.
11
12     double intToDouble = intNumber;
13     Console.WriteLine(intToDouble);
14 }
```

실행 결과

```
2147483647
2147483647
```

10 자료형 변환

■ 다른 자료형을 숫자로 변환

- 문자열과 불을 숫자 자료형으로 변환하려면 어떻게 해야 할까?

코드 2-63 문자열을 숫자로 변환

/2장/Casts

```
01 static void Main(string[] args)
02 {
03     // string 자료형을 int 자료형으로 변환합니다.
04     string numberString = "52273";
05     int intNumber = (int)numberString;
06     Console.WriteLine(intNumber);
07 }
```

- 이 코드는 오류가 발생해서 실행조차 되지 않음

10 자료형 변환

■ 다른 자료형을 숫자로 변환

- 다른 자료형을 숫자로 변환할 때는 [표 2-26]의 메서드를 사용

표 2-26 문자열을 숫자로 변환하는 메서드

메서드	설명
int.Parse()	다른 자료형을 int 자료형으로 변경합니다.
long.Parse()	다른 자료형을 long 자료형으로 변경합니다.
float.Parse()	다른 자료형을 float 자료형으로 변경합니다.
double.Parse()	다른 자료형을 double 자료형으로 변경합니다.

10 자료형 변환

■ 다른 자료형을 숫자로 변환

■ 기본예제 2-27 문자열을 숫자로 변환

/2장/StringTo

- 다음 코드는 문자열을 int, long, float, double 자료형으로 변환

코드 2-64 문자열을 숫자로 변환

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(int.Parse("52"));
04     Console.WriteLine(long.Parse("273"));
05     Console.WriteLine(float.Parse("52.273"));
06     Console.WriteLine(double.Parse("103.32"));
07
08     Console.WriteLine(int.Parse("52").GetType());
09     Console.WriteLine(long.Parse("273").GetType());
10     Console.WriteLine(float.Parse("52.273").GetType());
11     Console.WriteLine(double.Parse("103.32").GetType());
12 }
```

실행 결과

```
52
273
52.273
103.32
System.Int32
System.Int64
System.Single
System.Double
```

■ FormatException 예외

- Parse() 메서드를 사용할 때에 해당 자료형으로 변환 불가능한 문자를 넣으면 어떻게 될까?

코드 2-65

숫자로 변환할 수 없는 문자열을 변환하는 경우

/2장/Casts

```
01 Console.WriteLine(int.Parse("52.273"));  
02 Console.WriteLine(int.Parse("abc"));
```

- 이러한 예외가 발생하지 않게 Parse () 메서드의 매개변수에는 변환 가능한 문자열만 넣기 바람

■ 다른 자료형을 문자열로 변환

- 다른 자료형을 문자열로 변환할 때는 ToString() 메서드를 사용

표 2-27 문자열로 변환하는 메서드

메서드	설명
ToString()	문자열로 변환합니다.

- int 자료형을 문자열로 변환하고 싶다면 다음과 같은 코드를 사용

(10).ToString()

- double 자료형을 문자열로 변환하고 싶다면 다음과 같은 코드를 사용

(52.273).ToString()

■ 다른 자료형을 문자열로 변환

■ 기본예제 2-28 기본 자료형을 문자열로 변환

/2장

코드 2-66

기본 자료형을 문자열로 변환

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine((52).ToString());
04     Console.WriteLine((52.273).ToString());
05     Console.WriteLine(('a').ToString());
06     Console.WriteLine((true).ToString());
07     Console.WriteLine((false).ToString());
08
09     Console.WriteLine((52).ToString().GetType());
10     Console.WriteLine((52.273).ToString().GetType());
11     Console.WriteLine(('a').ToString().GetType());
12     Console.WriteLine((true).ToString().GetType());
13     Console.WriteLine((false).ToString().GetType());
14 }
```

실행 결과

```
52
52.273
a
True
False
System.String
System.String
System.String
System.String
System.String
```

10 자료형 변환

■ 다른 자료형을 문자열로 변환

■ 기본예제 2-29 소수점 제거

/2장/DoubleToString

- ToString() 메서드를 사용하면 실수의 소수점을 원하는 곳까지만 출력할 수 있음

코드 2-67 소수점 제거

```
01 static void Main(string[] args)
02 {
03     double number = 52.273103;
04     Console.WriteLine(number.ToString("0.0"));
05     Console.WriteLine(number.ToString("0.00"));
06     Console.WriteLine(number.ToString("0.000"));
07     Console.WriteLine(number.ToString("0.0000"));
08 }
```

실행 결과

```
52.3
52.27
52.273
52.2731
```

■ 다른 자료형을 문자열로 변환

■ 기본예제 2-30 숫자와 문자열 덧셈

/2장/StringPlusNumber

- 숫자와 문자열을 더할 수 있는 모든 경우를 나열하고 출력
- 숫자와 문자열을 놓고 + 기호를 사용하면 문자열 연결 연산자로 적용됨

코드 2-68 숫자와 문자열 덧셈

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(52 + 273);
04     Console.WriteLine("52" + 273);
05     Console.WriteLine(52 + "273");
06     Console.WriteLine("52" + "273");
07 }
```

실행 결과

```
325
52273
52273
52273
```

■ 간단한 문자열 변환

- 문자열과 다른 자료형을 더하면 문자열 연결을 수행하게 됨

코드 2-69

간단한 문자열 변환

/2장/Casts

```
01 int number = 52273;  
02 string outputA = number + "";  
03 Console.WriteLine(outputA);  
04  
05 char character = 'a';  
06 string outputB = character + "";  
07 Console.WriteLine(outputB);
```

- 문자는 문자열과 비슷하니까 바로 문자열로 변환할 수 있다고도 생각할 수 있는데, 불가능

```
char character = 'a';  
string output = character;
```

☞ (지역 변수) char character

CS0029: 암시적으로 'char' 형식을 'string' 형식으로 변환할 수 없습니다.

그림 2-23 문자 문자열 변환 오류

10 자료형 변환

■ 다른 자료형을 불로 변환

- 문자열을 불 자료형으로 변환할 때는 [표 2-28]의 메서드를 사용

표 2-28 문자열을 불로 변환하는 메서드

메서드	설명
<code>bool.Parse()</code>	문자열을 불 자료형으로 변환합니다.

10 자료형 변환

■ 다른 자료형을 불로 변환

■ 기본예제 2-31 문자열을 불로 변환

/2장/StringToBool

코드 2-70 문자열을 불로 변환

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine(bool.Parse("True"));
04     Console.WriteLine(bool.Parse("true"));
05     Console.WriteLine(bool.Parse("False"));
06     Console.WriteLine(bool.Parse("false"));
07 }
```

실행 결과

```
True
True
False
False
```

■ 음수밖에 없는 숫자

- int 자료형의 범위는 -2147483648부터 2147483647까지인데, 결국 -2147483648에 대응되는 양수가 없음
- -2147483648에는 음수 변환 연산자(-)를 붙여도 자기 자신이 됨

코드 2-71

int 자료형 최솟값의 음수

```
01 static void Main(string[] args)
02 {
03     int output = int.MinValue;
04     Console.WriteLine(-output);
05 }
```

실행 결과

-2147483648

■ 음수밖에 없는 숫자

- 다음과 같은 코드는 오류가 발생

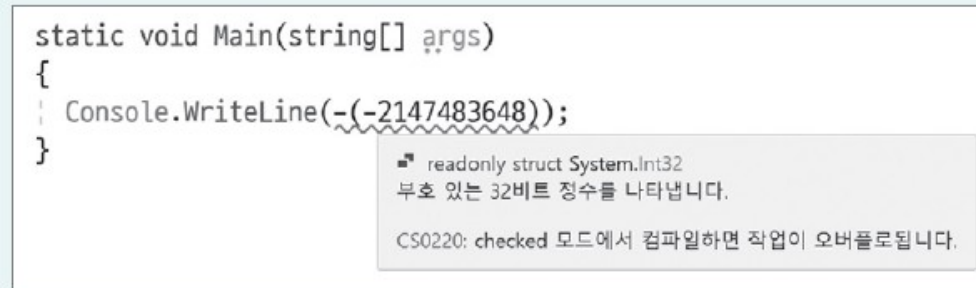


그림 2-24 숫자를 직접 입력하면 오류 발생

- int 자료형을 사용하는 프로그래밍 언어(C, C++, 자바) 등에서는 모두 이와 비슷한 현상이 발생